# Nephtys: Lightweight Edge Connector for Bandwidth-Efficient Ingestion of Urban Sensor Streams

Andrea Bozzo
Independent Researcher
andreabozzo92@gmail.com

***Abstract*—Smart City sensor networks generate continuous telemetry that must be ingested, filtered, and forwarded by resource-constrained edge gateways. Traditional message brokers are too heavy for edge deployment, while lightweight alternatives lack processing capabilities. We present Nephtys, a Go-based edge connector that ingests real-time data streams from heterogeneous sources (WebSocket, REST, SSE, gRPC) and processes them through per-stream middleware pipelines, i.e. composable chains of transform, deduplication, threshold filtering, and batching stages, that can be hot-swapped at runtime via a declarative REST API. Using NATS JetStream as the sole persistence and transport layer, Nephtys achieves zero-infrastructure state recovery with no external databases. We evaluate the system on an urban air-quality monitoring scenario with 20 simulated sensor stations and five real data feeds from the Open-Meteo and Sensor.Community APIs. Results on both synthetic and real data (Open-Meteo, Sensor.Community) show 69–84% bandwidth reduction and up to 98.8% message-count reduction within a 28.6 MB memory footprint, with sub-16 ms pipeline hot-reload latency and automatic reconnection after network failures.**

***Index Terms*—Edge computing, IoT, Smart City, data ingestion, stream processing, NATS, middleware pipeline**

## I. Introduction

Smart City deployments and the Industrial Internet of Things (IIoT) have shifted data acquisition from static, batch-oriented collection to continuous, high-frequency streaming from ubiquitous sensor networks [1]. Urban air-quality monitoring stations, traffic sensors, and environmental probes generate telemetry at rates that quickly saturate the wide-area links connecting edge gateways to centralized cloud backends. At the same time, traditional message-brokering infrastructures (predominantly JVM-based systems such as Apache Kafka [2] or RabbitMQ) impose memory and CPU requirements that exceed the budget of resource-constrained edge nodes [3].

Existing lightweight alternatives, such as MQTT bridges [4] or protocol translators, provide raw message forwarding but lack the ability to *process* data at the edge. Conversely, full-fledged stream-processing frameworks (e.g., Apache Flink, Kafka Streams) are designed for cluster-level deployment and are impractical on single-board computers or micro-gateways [5]. This leaves an open problem: no lightweight, edge-deployable solution currently supports **runtime-reconfigurable processing pipelines**, i.e. the ability to inject, modify, or remove filtering, deduplication, and transformation logic on a live data stream without service interruption.

This paper presents **Nephtys**, a telemetry ingestion connector developed in Go [6] that addresses this problem. Nephtys runs on constrained edge hardware and ingests real-time data streams from heterogeneous sources (WebSocket, REST polling, Server-Sent Events, Webhooks, gRPC). Each stream passes through a per-stream **middleware pipeline**: a composable chain of filter, transform, deduplicate, threshold, enrich, and batch stages that can be injected or hot-swapped at runtime via a declarative REST API without restarting the stream. Processed events are published to NATS JetStream [7], which also serves as the sole persistence layer: stream configurations are stored in a JetStream Key-Value bucket, enabling zero-infrastructure state recovery after crashes or reboots.

The contributions of this work are as follows:

1) A **multi-protocol ingestion architecture** with native fault tolerance (exponential-backoff reconnection, graceful shutdown) designed for intermittent edge connectivity.
2) A **dynamically reconfigurable middleware pipeline** engine that reduces upstream bandwidth by filtering, deduplicating, and compacting sensor payloads at the edge, demonstrated with 69–84% bandwidth reduction on both synthetic and real urban air-quality streams.
3) A **zero-infrastructure persistence** model that eliminates the need for external databases on the edge, relying exclusively on the NATS JetStream subsystem for both event durability and configuration state.

## II. System Architecture and Resilience

Nephtys targets the constraints of edge computing: limited memory, unreliable network connectivity, and the absence of orchestration infrastructure [3]. Fig. 1 illustrates the overall architecture: urban sensor stations feed data into Nephtys through protocol-specific connectors; each

stream traverses a configurable middleware pipeline before publication to NATS JetStream [7], from which cloud-side consumers subscribe.

### A. Fault-Tolerant Ingestion

Each data source is managed by a *connector* abstraction that encapsulates the protocol-specific logic (WebSocket, REST poller, SSE, webhook, gRPC). In a Smart City deployment [1], connectors may simultaneously ingest high-frequency air-quality readings via WebSocket from a local sensor gateway and poll a municipal open-data REST API for meteorological context.

All connectors implement automatic reconnection with **exponential backoff** (initial delay 1 s, capped at 30 s). This prevents accidental denial-of-service flooding when an upstream sensor gateway reboots or loses connectivity, a common occurrence on battery-powered or solar-powered field nodes. On connection loss the connector transitions to a *reconnecting* state and resumes ingestion transparently once the source is available again.

### B. Zero-Infrastructure Persistence

Edge nodes frequently lack the resources to run a relational database or even an embedded key-value store alongside the application. Nephtys addresses this by using **NATS JetStream** [7] as both the event transport and the configuration store. Published sensor events are stored in a durable JetStream stream with configurable retention (default: 72 hours), surviving process restarts without data loss. Active stream configurations are serialized in a JetStream Key-Value bucket (`nephtys_streams`). On startup, the Stream Manager reads all persisted entries and re-registers the corresponding connectors and pipelines, achieving full state recovery with no external dependencies.

This design means Nephtys and NATS are the *only two processes* required on an edge node, a minimal footprint suitable for single-board computers such as the Raspberry Pi 4. All connectors support TLS-encrypted upstream connections (WSS, HTTPS), and the management REST API is protected by Bearer-token authentication, preventing unauthorized pipeline reconfiguration.

### C. Edge Pipeline for Bandwidth Reduction

The central component is the **Pipeline Middleware** engine. Before an event reaches the broker, it traverses a chain of composable middleware stages, each independently configurable per stream:

- **Transform**: Extracts only the relevant fields from nested JSON payloads using dot-notation path mapping, reducing per-event payload size.
- **Dedup**: Computes the FNV-64a hash [8] of each payload and blocks duplicates within a configurable time-to-live window.
- **Threshold**: Passes an event only if the absolute change in a numerical field exceeds a configurable delta (e.g., $\Delta \mathrm{PM}_{2.5} \geq 1.0\ \mu\mathrm{g/m}^3$), filtering out sensor noise.
- **Batch**: Buffers events and flushes them as a single aggregated message at a configurable interval or batch size, reducing per-message overhead.
- **Filter** and **Enrich**: Drop events not matching specified types, or inject static metadata tags.

The pipeline configuration is a JSON object attached to each stream registration and can be **hot-swapped at runtime** via a `PUT /v1/streams/{id}/pipeline` REST call. Internally, the active pipeline handler is stored behind a Go `atomic.Pointer`, so the swap is lock-free and takes effect on the next event with sub-millisecond latency. This allows operators to tighten or relax filtering thresholds in response to air-quality alerts without interrupting data flow.

## III. Evaluation Scenario

We evaluate Nephtys in an urban air-quality monitoring scenario representative of a Smart City edge deployment. All source code, the sensor simulator, and an automated benchmark script are publicly available in the companion repository[1] to allow full reproducibility of the results presented below.

### A. Experimental Setup

Nephtys runs on a host machine; NATS JetStream, Prometheus, and Grafana are co-deployed via Docker Compose. Two classes of data sources feed the system:

1) **Synthetic sensor streams**: A lightweight WebSocket-based simulator (`sensor-sim`) generates air-quality readings for 20 virtual stations at 2 Hz, producing ∼40 events/s. Each event contains $\mathrm{PM}_{2.5}$, $\mathrm{PM}_{10}$, $\mathrm{NO}_2$, temperature, and humidity values with configurable jitter and a 30% duplicate ratio, modeling the redundancy typical of low-cost particulate-matter sensors.
2) **Real-world open data**: Five REST poller streams ingest live measurements from the Open-Meteo weather and air-quality APIs [9] for three cities in Calabria (Cosenza, Catanzaro, Reggio Calabria) and from a Sensor.Community [10] citizen PM sensor near Gioia Tauro, polled every 30–60 seconds.

The full middleware pipeline is configured as: Transform (5 fields extracted), Dedup (TTL = 30 s, cache = 500), Threshold (path = `pm25`, $\delta = 1.0\ \mu\mathrm{g/m}^3$), Batch (size = 50, flush = 5 s).

### B. Results

Table I reports metrics from two experiments: (a) a 5-minute synthetic run with 20 virtual stations at 2 Hz (∼40 events/s, 30% duplicate ratio), and (b) a 30-minute real-data run with five REST poller streams hitting the Open-Meteo [9] and Sensor.Community [10] APIs.

[1]Companion material: https://github.com/AndreaBozzo/uic2026-nephtys; Nephtys source: https://github.com/AndreaBozzo/Nephtys

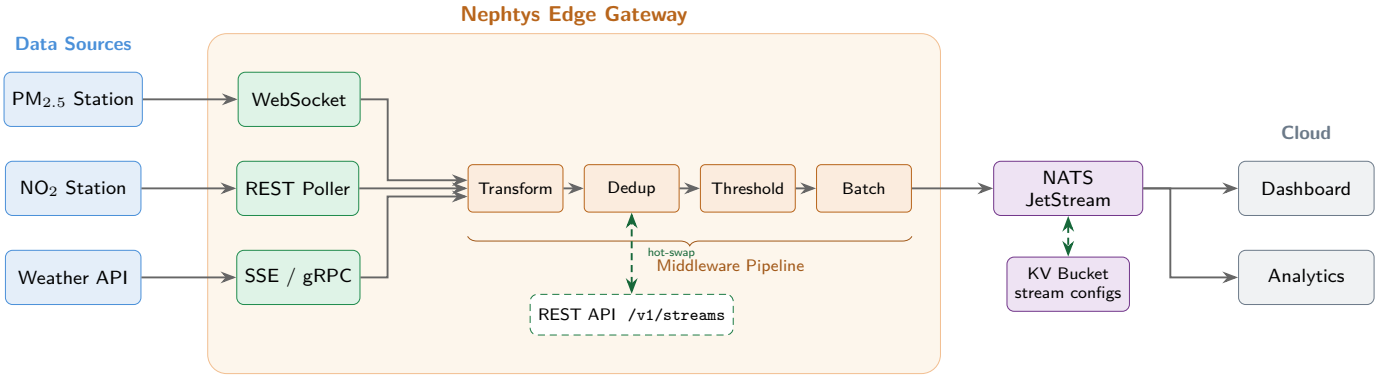


Fig. 1. Nephtys system architecture. Sensor data flows left-to-right through protocol connectors and a composable middleware pipeline before publication to NATS JetStream. The REST API enables runtime hot-swapping of pipeline configurations.

TABLE I
BENCHMARK RESULTS: BASELINE VS. FULL PIPELINE ON SYNTHETIC AND REAL-DATA STREAMS.

| Metric | Baseline | Pipeline | Reduction |
|---|---|---|---|
| *Synthetic streams (20 stations, 5 min)* | | | |
| Bytes published | 2,759,750 | 856,228 | 69.0% |
| Messages published | 12,120 | 149 | 98.8% |
| *Real-data streams (5 APIs, 30 min)* | | | |
| Bytes published | 146,931 | 23,653 | 83.9% |
| Messages published | 275 | 30 | 89.1% |
| RSS memory | 28.6 MB | | — |

**Synthetic workload.** The bandwidth reduction is decomposed by middleware stage using the Prometheus counter `events_dropped_by_pipeline_total`. Of the 12,120 events ingested, **Dedup** dropped 3,713 (78.7% of all drops), i.e. redundant identical readings within the 30 s TTL window. **Threshold** filtered an additional 1,005 events (21.3% of drops), corresponding to sub-delta $PM_{2.5}$ fluctuations. The remaining 7,402 events were compacted by **Batch** into 149 messages (avg. 50 events per message), and **Transform** reduced the per-event payload from 227 to 116 bytes (49% reduction).

**Real-data workload.** On the five live API streams polled over 30 minutes, 155 of 275 events were dropped by dedup and 90 by threshold, reducing published messages to 30 (89.1%). The byte reduction was even higher (83.9%) because Transform extracts 5–6 fields from the ∼465-byte Open-Meteo JSON responses. The four Open-Meteo streams each achieved >99% bandwidth reduction; the Sensor.Community stream, which reports genuinely new readings every ∼70 s, showed a lower but still meaningful 21% reduction via dedup alone.

**Resilience.** When the sensor simulator process is killed, the WebSocket connector detects the disconnection and enters exponential backoff (1 s, 2 s, 4 s, . . . , capped at 30 s). After the simulator is restarted, Nephtys reconnects automatically within 1–2 s. No crash or manual intervention is required; ingestion simply pauses and resumes.


Fig. 2. Grafana dashboard during a mixed workload (synthetic + real-data streams), showing ingested vs. published rates, per-middleware drop breakdown, and reduction gauges (65.7% bandwidth, 98.6% events).

**Hot-reload latency.** Issuing a `PUT` request to change the threshold delta completes in under 16 ms (including the full HTTP round-trip). The atomic pointer swap ensures the new pipeline takes effect on the very next event, with no loss or reordering.

Fig. 2 shows the Grafana dashboard captured during the benchmark, displaying the real-time bandwidth and event reduction ratios.

## IV. Related Work and Conclusions

### A. Related Work

Several frameworks target data processing at the IoT edge, but each carries trade-offs that Nephtys addresses. **EdgeX Foundry** [11] is a comprehensive, microservice-based edge platform whose multi-container deployment model imposes considerable resource overhead (typically > 512 MB RAM). **Eclipse Kura** [12] provides an OSGi-based Java runtime for IoT gateways, but its JVM dependency conflicts with the minimal-footprint goal. **MQTT bridges** [4] offer lightweight forwarding but no processing capability. **Node-RED** [13] enables visual flow-based programming for IoT, but its Node.js runtime results in higher memory consumption and lower throughput.

Nephtys differentiates itself through the combination of (i) a compiled, single-binary Go [6] deployment with $< 30$ MB RSS, (ii) a per-stream middleware pipeline that is hot-swappable at runtime via REST API, and (iii) zero-infrastructure persistence using NATS JetStream [7] as the sole backing store.

### B. Conclusions

This paper presented Nephtys, a lightweight edge connector that demonstrates how compiled languages and lightweight message brokers can enable bandwidth-efficient telemetry ingestion for Smart City sensor networks. By shifting deduplication, transformation, and threshold filtering to the edge through declaratively configurable middleware pipelines [5], the system achieves 69–84% reduction in upstream bandwidth and up to 98.8% reduction in message count on both synthetic and real urban air-quality workloads, all within a memory footprint of 28.6 MB.

The runtime reconfigurability of the pipeline, enabled by atomic pointer swapping, allows operators to adapt filtering behavior to changing conditions (e.g., tightening thresholds during pollution alerts) without interrupting data flow, a capability absent from existing lightweight edge solutions.

Future work includes the addition of an MQTT connector to support legacy sensor deployments, the integration of lightweight ML-based anomaly detection as a pipeline middleware stage, and the federation of multiple Nephtys instances across a city-wide edge mesh for coordinated, distributed intelligence at the urban periphery.

## References


[1] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, "Internet of things for smart cities," *IEEE Internet of Things Journal*, vol. 1, no. 1, pp. 22–32, 2014.

[2] J. Kreps, N. Narkhede, and J. Rao, "Kafka: a distributed messaging system for log processing," in *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*, Athens, Greece, 2011, pp. 1–7.

[3] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[4] A. Banks, E. Briggs, K. Borgendale, and R. Gupta, "MQTT version 5.0," OASIS, OASIS Standard, 2019, 07 March 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html

[5] M. D. de Assunção, A. da Silva Veith, and R. Buyya, "Distributed data stream processing and edge computing: A survey on resource elasticity and future directions," *Journal of Network and Computer Applications*, vol. 103, pp. 1–17, 2018.

[6] The Go Authors, "The Go programming language specification," https://go.dev/ref/spec, 2024, accessed: 2026-03-26.

[7] NATS Authors, "NATS – the cloud native messaging system," https://nats.io, 2024, accessed: 2026-03-26.

[8] G. Fowler, L. C. Noll, K.-P. Vo, D. E. Eastlake, III, and T. Hansen, "The FNV non-cryptographic hash algorithm," IETF Internet-Draft, https://datatracker.ietf.org/doc/html/draft-eastlake-fnv-17, 2019, informational.

[9] P. Zippenfenig, "Open-Meteo – free weather API," https://open-meteo.com, 2024, accessed: 2026-03-26.

[10] Sensor.Community, "Sensor.Community – a contributors driven global sensor network," https://sensor.community, 2024, accessed: 2026-03-26.

[11] LF Edge, "EdgeX Foundry – open-source edge computing platform for IoT," https://www.edgexfoundry.org, 2024, accessed: 2026-03-26.

[12] Eclipse Foundation, "Eclipse Kura – the extensible open source Java/OSGi IoT edge framework," https://eclipse.dev/kura/, 2024, accessed: 2026-03-26.

[13] OpenJS Foundation, "Node-RED – low-code programming for event-driven applications," https://nodered.org, 2024, accessed: 2026-03-26.